ZAP IDE Integration Guide

The author of this software stands in solidarity with 🇺🇦 Ukraine. We believe in a world where international borders are respected and human rights are upheld. We encourage all users of this software to contribute to humanitarian efforts in 🇺🇦 Ukraine.

VS Code Extension for the ZAP Programming Language

Version: 1.1 **Date**: March 2026

Table of Contents

Installation

Prerequisites

- **VS Code** 1.70 or later
- **zapc** compiler in your PATH (required for inline diagnostics and build tasks)

Install the Extension

From a local build:

```
cd IDE_Integration
./install_vscode_extension.sh       # Linux / macOS
install_vscode_extension.bat        # Windows (CMD)
install_vscode_extension.ps1        # Windows (PowerShell)
```

After running the script, **reload VS Code** (`Ctrl+Shift+P` → *Reload Window*).

Verify installation: Open any `.zap` file — you should see syntax colors and the ZAP icon in the file tab.

Quick Start

1. Open a `.zap` file in VS Code.
2. The extension activates automatically.
3. Error squiggles appear within 1.5 s if there are compilation errors.
4. Press `Ctrl+Shift+Z` to compile and see output in the terminal.

Feature Reference

Syntax Highlighting

ZAP source files (`.zap`) receive full colorization:

- **Keywords** — `proc`, `func`, `if`, `while`, `for`, `repeat`, `switch`, `struct`, `enum`, `const`, `asm`, etc.
- **Types** — `byte`, `word`, `long` and pointer modifier `^`
- **Literals** — decimal (`123`), hex (`$FF`), binary (`%1010`), strings (`"text"`), chars (`'a'`)
- **Operators** — arithmetic, bitwise, comparison, address-of (`@`), member access (`.`)
- **Preprocessor directives** — `.include`, `.define`, `.ifdef`, `.ifndef`, `.module`, etc.
- **Declaration modifiers** — `#PORT`, `#RD`, `#WR`, `#KEEP`, `#NOEXPORT`, `#EXPORT`
- **Inline assembly** — `asm … end` blocks use embedded ca65 assembly syntax highlighting

---

Code Folding

Click the fold gutter (left of line numbers) to collapse any block:

- `proc … end`, `func … end`
- `if … end`, `while … end`, `for … end`, `repeat … until`
- `switch … end` (also folds individual `case`/`default` → `break` sections)
- `struct … end`, `enum … end`
- `asm … end`
- Block comments `/* … */`

Struct Member Completions

When you type `.` after a variable that has a struct type, a completion popup shows all fields of that struct.

Example:

```
FILE fd

rv = fopen(@fd, FName, ICAX1_Mode.Write)
fd.           ; ← type dot here
```

The popup shows:

```
fd      byte   — handle
error   ERRNO  — error code
eof     BOOL   — EOF reached
```

Selecting a field inserts only the field name — the dot you just typed is kept.

How it works: The extension scans the current file and all `.included` files for `FILE fd` to determine the type, then finds `struct FILE … end` and lists its fields.

Enum Member Completions

When you type `.` after an enum type name, all members with their values appear.

Example:

```
ICAX1_Mode.   ; ← type dot here
```

The popup shows:

```
Read       = 4
Directory  = 6
Write      = 8
Append     = 9
ReadWrite  = 12
```

This works for any enum defined in the current file or its includes. Both decimal and hex values (`$03`) are handled.

Identifier Completions

While typing any identifier, VS Code automatically shows a completion list of all known symbols from the current file and all included files.

What is listed:

Symbol kind	Example detail shown
Variable	`byte rv`
Variable (struct)	`FILE fd`
Constant	`const byte FILE_HANDLE_MAX`
Procedure	`proc fopen(byte^ fd, byte^ name, byte mode)`
Function	`func byte strlen(byte^ str)`
Struct type	`struct FILE`
Enum type	`enum ICAX1_Mode`

Selecting a **procedure or function** inserts a call snippet with tab stops on each argument:

```
fopen(|fd|, |name|, |mode|)
```

Hover Information

Hover the mouse over any identifier (without clicking) to see a tooltip.

Variable:

```
FILE fd

Fields:
• byte fd — handle
• ERRNO error — error code
• BOOL eof — EOF reached
```

Procedure:

```
proc fopen(byte^ fd, byte^ name, byte mode)
```

Function with defaults:

```
func byte CIO(byte ch, byte command, word adr = 0, word len = 0,
              byte aux1 = 0, byte aux2 = 0, byte aux3 = 0)
```

Struct type:

```
struct FILE

Fields:
• byte fd — handle
• ERRNO error — error code
• BOOL eof — EOF reached
```

Enum type:

```
enum ICAX1_Mode

Members:
• Read = 4
• Write = 8
• Append = 9
```

Member access — hover on `error` in `fd.error`:

```
FILE.error : ERRNO
```

Enum member — hover on `Write` in `ICAX1_Mode.Write`:

```
ICAX1_Mode.Write = 8
Member of enum ICAX1_Mode
```

Signature Help

When you type `(` after a procedure or function name, a tooltip shows the full signature. The **current parameter is highlighted** in bold. Typing `,` advances to the next parameter.

Example:

```
rv = fopen(
;   ┌──────────────────────────────────────────────┐
;   │ proc fopen(byte^ fd, byte^ name, byte mode) │
;   │             ^^^^^^^^                         │
;   └──────────────────────────────────────────────┘
```

After the first comma:

```
;  proc fopen(byte^ fd, byte^ name, byte mode)
;                      ^^^^^^^^^
```

Works for all procedures and functions, including those with default parameter values.

Go to Definition

Press **F12** (or **Ctrl+Click**) on any identifier to jump to where it is declared.

What you click	Where you land
`fopen`	`proc fopen(...)` in `atari_stdio.zap`
`fd`	`FILE fd` declaration in the current file
`FILE`	`struct FILE` definition in `types.zap`
`ICAX1_Mode`	`enum ICAX1_Mode` in `atari_stdio.zap`
`error` in `fd.error`	`error` field line inside `struct FILE`
`Write` in `ICAX1_Mode.Write`	`Write = 8` line inside `enum ICAX1_Mode`

If the declaration is in an included file, VS Code opens that file and scrolls to the line automatically.

Tip: Use **Alt+Left** (Back) to return to where you came from after jumping.

Find All References

Press **Shift+F12** (or right-click → **Find All References**) to list every use of an identifier across all files.

Plain identifier (e.g., `fopen`): Lists every call site in the current file and all included files. The declaration line itself is excluded from the list (shown separately in the peek view header).

Member access — cursor on `error` in `fd.error`: Lists every `.error` member access anywhere in the codebase.

Results appear in the References panel at the bottom; clicking any entry navigates to it.

Document Symbols / Outline

Open the **Outline** panel with **View** → **Open View** → **Outline** (or the Explorer sidebar's Outline section).

The outline shows a full tree of the current file's symbols:

```
▶ FILE                  struct
▶ ICAX1_Mode            enum
▶ CONSTRUCTOR           proc  ()
▶ puts                  proc  (byte^ str)
```

```
▶ fopen                  proc  (byte^ fd, byte^ name, byte mode)
    fd                     TypeParameter
    name                   TypeParameter
    mode                   TypeParameter
▶ strlen                 func  byte (byte^ str)
    str                    TypeParameter
```

Clicking any entry jumps directly to that symbol.

The **breadcrumb bar** at the top of each editor tab shows which symbol the cursor is currently inside:

```
atari_stdio.zap  >  fopen  >  ...
```

Inline Compiler Diagnostics

Errors reported by `zapc` appear as **red squiggles** directly in the editor — no need to open a terminal.

Trigger conditions:

Event	Delay
File opened	Immediate
File saved (`Ctrl+S`)	Immediate
Typing stopped	1.5 s after last keystroke

Reading errors:

- Hover over a squiggle to read the full error message.
- The **Problems panel** (`Ctrl+Shift+M`) lists all errors with file, line, and column.
- Errors in **included files** appear in those files directly when they are open.

Example:

```
byte rv
rv = unknownFunc()   ; ← red squiggle: "Undefined function: unknownFunc"
```

> **Note:** Inline diagnostics require `zapc` to be accessible in your system `PATH`. If `zapc` is not found, no squiggles appear, but all other features (completions, hover, navigation) continue to work.

AI Inline Code Completions

The extension supports AI-powered inline code suggestions (ghost text), similar to GitHub Copilot but tailored specifically for ZAP!. The AI understands ZAP! syntax, types, control flow, and 6502/65C02 conventions.

Setup

1. **Get an API key** from Anthropic or OpenAI.
2. **Configure settings** in VS Code (`File → Preferences → Settings`, search for "zap.ai"):

Setting	Default	Description
`zap.ai.enabled`	`false`	Enable/disable AI completions
`zap.ai.provider`	`anthropic`	AI provider: `anthropic` or `openai`
`zap.ai.apiKey`	(empty)	Your API key
`zap.ai.model`	`claude-sonnet-4-6-20250514`	Model name
`zap.ai.endpoint`	(empty)	Custom API endpoint (for proxies or local models like Ollama)
`zap.ai.maxTokens`	`256`	Max tokens per completion
`zap.ai.debounceMs`	`500`	Delay before requesting a completion

3. **Enable** by setting `zap.ai.enabled` to `true`, or click the **"ZAP AI: Off"** status bar item to toggle.

Usage

Once configured, ghost text suggestions appear automatically as you type inside a `.zap` file:

- **Type a partial line** and pause — a gray suggestion appears after the debounce delay.
- Press **Tab** to accept the suggestion.
- Press **Escape** to dismiss it.
- The status bar shows the current state: `$(sparkle) ZAP AI: On`, `$(circle-slash) ZAP AI: Off`, or `$(warning) ZAP AI: No Key`.
- A spinning icon appears while a request is in flight.

What the AI knows about ZAP!:

- All ZAP! syntax: `proc`, `func`, `if/else/end`, `while`, `for`, `repeat/until`, `switch/case`
- Data types: `byte`, `word`, `long`, pointers, structs, enums, arrays
- Built-in functions: `PEEK()`, `POKE()`, `LOW()`, `HIGH()`, `SIZEOF()`
- 6502/65C02 target conventions (memory-conscious, small values)

The AI reads the surrounding code context (up to 100 lines before and 30 lines after the cursor) to generate relevant suggestions.

Supported Providers

Provider	Endpoint	Models
Anthropic	`https://api.anthropic.com`	`claude-sonnet-4-6-20250514`, `claude-haiku-4-`

Provider	Endpoint	Models
		`5-20251001`, etc.
OpenAI	`https://api.openai.com`	`gpt-4o`, `gpt-4o-mini`, etc.
Ollama (local)	Set `zap.ai.endpoint` to `http://localhost:11434` and `zap.ai.provider` to `openai`	Any Ollama model

> **Note:** AI completions are optional and disabled by default. The extension works fully without them. API usage incurs costs based on your provider's pricing.

Code Snippets

Type a short prefix and press **Tab** to insert a full code template. Tab stops let you fill in each placeholder in order, pressing **Tab** again to advance.

Control Flow

`if` → if block:

```
if condition
    |cursor|
end
```

`ife` → if/else:

```
if condition

else
    |cursor|
end
```

`ifee` → if/elseif/else:

```
if condition

elseif condition2

else
    |cursor|
end
```

`while` → while loop:

```
while condition
    |cursor|
end
```

`for` → **for loop:**

```
for i = 0 to N
    |cursor|
end
```

`ford` → **for downto:**

```
for i = N downto 0
    |cursor|
end
```

`fors` → **for with step:**

```
for i = 0 to N step 2
    |cursor|
end
```

`repeat` → **repeat/until:**

```
repeat
    |cursor|
until condition
```

`switch` → **switch with case and default:**

```
switch variable
    case value
        |cursor|
    break
    default

    break
end
```

`case` → **single case block:**

```
case value
    |cursor|
break
```

Declarations

`proc` → procedure:

```
proc name()
    |cursor|
end
```

`func` → function:

```
func byte name()
    |cursor|
    return 0
end
```

`struct` → struct type:

```
struct Name
    byte field
end
```

`enum` → enum type:

```
enum Name
    Member1,
    Member2
end
```

`asm` → inline assembly block:

```
asm
    |cursor|
end
```

Variables and Constants

Prefix	Expands to
`byte`	`byte name = 0`
`word`	`word name = 0`
`long`	`long name = 0`
`bytearr`	`byte name[size]`
`wordarr`	`word name[size]`
`constb`	`const byte NAME = 0`
`constw`	`const word NAME = 0`
`consts`	`const byte NAME[] = "text"`

Preprocessor

Prefix	Expands to
`.module`	`.module "name"`
`.include`	`.include "file.zap"`
`.define`	`.define SYMBOL`
`.ifdef`	`.ifdef SYMBOL … .endif`
`.ifndef`	`.ifndef SYMBOL … .endif`

Build Integration

Shortcut	Action
Ctrl+Shift+Z	Compile current file — output appears in the *ZAP Compiler* terminal panel
Ctrl+Alt+Z	Run the *zap: Build ZAP project* task
Ctrl+Shift+B	Open VS Code build task picker

Errors from the build appear in the **Problems** panel and, if the file is open, as inline squiggles.

Right-click a `.zap` file in the Explorer → **ZAP: Compile current file** also works.

Keyboard Shortcuts

Key	Action
Ctrl+Shift+Z	Quick compile
Ctrl+Alt+Z	Build task
F12	Go to Definition

Key	Action
Shift+F12	Find All References
Ctrl+Click	Go to Definition
Alt+F12	Peek Definition (inline)
Alt+Left	Navigate back
Ctrl+Space	Force open completion popup
Ctrl+Shift+M	Open Problems panel
Tab (after snippet prefix)	Expand snippet

Cross-File Support

All IntelliSense features follow `.include` directives recursively. When your file contains:

```
.include "lib/stdio.zap"
.include "lib/types.zap"
```

…the extension automatically scans those files (and their own includes) for symbol definitions. This means:

- Completions show all library functions and types
- Hover shows signatures of library procs/funcs
- F12 can navigate into library files
- Find References searches across all included files

Circular includes are handled safely — each file is scanned at most once.

Case Insensitivity

ZAP is a **case-insensitive** language. The IDE features respect this fully:

- `FILE`, `file`, and `File` all resolve to the same struct
- `fopen`, `FOPEN`, and `Fopen` all resolve to the same procedure
- Completions, hover, go-to-definition, and find-references all work regardless of how you type identifiers

---

Troubleshooting

No error squiggles appear

- Make sure `zapc` is in your `PATH`: open a terminal and type `zapc --version`
- Check the **Output** panel (`View → Output`) and select "ZAP" from the dropdown for diagnostic messages

Completions don't show library symbols

- Verify that `.include "lib/..."` paths are correct and the files exist
- Save the file first — the extension rescans on each save

Go to Definition opens the wrong file

- This can happen if the same symbol name is declared in multiple included files. The extension returns the first definition found in include order.

Squiggles appear at wrong positions

- Compile the file explicitly with `Ctrl+Shift+Z` to see the exact error in the terminal with correct file/line/column information.

Extension not activating

- Open a `.zap` file (the extension activates on `.zap` file open)
- Check **Extensions** panel (`Ctrl+Shift+X`) that *ZAP Language Support* shows as enabled
- Run *Reload Window* (`Ctrl+Shift+P` → Reload Window) after installation